



N C B W[®]
1 0 0

**National Coalition of 100 Black Women
Leadership Academy Tracking Software
System Manager Documentation**

April 2026
Leadership Academy Tracking Software v1.0

Table of Contents

1.	Introduction	5
1.1	Purpose of the System	5
1.2	Description of the Current System	5
1.3	Scope of the System	6
1.4	Fundamental Assumptions	6
1.5	System Fit and Organizational Context	6
1.6	Benefits of the System	7
1.7	Design Goals	7
1.8	References	8
2.	System Requirements Specification	9
2.1	User Characteristics	9
2.2	Functional Requirements	10
2.3	Nonfunctional Requirements	17
3.	Access Control and Security Decisions	19
3.1	Access Matrix	19
3.2	Authentication Mechanism	19
3.3	Data Encryption	19
3.4	Security and Privacy	20
4.	Design Decisions	20
4.1	User Interface	20
4.2	Data Presentation	20
4.3	Exception Handling	21
4.4	Languages and Rules	21
5.	Architectural/Detailed Design	22

5.1	System Overview	22
5.2	Subsystem Design	23
5.3	Persistent Data Management	25
5.4	Project Structure	28
6.	Requirements Traceability Matrix	29
7.	System Deployment	30
7.1	System Requirements	30
7.2	Repository Setup	30
7.3	Backend Setup	30
7.4	Frontend Setup	31
7.5	Running the Full System	31
8.	System Configuration & Administration	32
8.1	System Access and Roles	32
8.2	Course and Certificate Management	32
8.3	System Configuration and Scheduled Tasks	33
9.	System Operations	35
9.1	System Startup and Shutdown	35
9.2	User Workflows	35
9.3	Data Backup	36
9.4	Monitoring and Logging	36
10.	Error Handling & Troubleshooting	37
10.1	Error Handling Approach	37
10.2	Logging and Error Reporting	37
10.3	Common Issues and Resolutions	37
10.4	Recovery Procedures	38

11.	Maintenance & Future Enhancements	38
11.1	System Maintenance	38
11.2	Routine Maintenance Tasks	38
11.3	Future Enhancements	38
12.	Testing & Validation Summary	39
12.1	Testing Approach	39
12.2	Types of Testing Performed	39
12.3	Results and Validation	39
13.	Glossary	40
14.	Appendix	42
A	Project Folder Structure	42
B	Database Schema	43

1. Introduction

1.1 Purpose of the System

The purpose of the Leadership Academy Tracking Software is to provide an efficient, centralized, and automated system for managing the leadership development programs of the National Coalition of 100 Black Women, Queen City Metropolitan Chapter.

The current manual tracking process using spreadsheets, email chains, and physical documents creates inefficiencies and increases the likelihood of errors. The proposed software addresses these challenges by:

- Automating the tracking of member progress across various leadership training tracks and modules.
- Generating and sending certificates of completion automatically.
- Providing administrators with real-time access to progress reports and data summaries.
- Sending automated reminders to trainees who are inactive for specific periods.

This software ensures that leadership training is streamlined, organized, and easily accessible for both members and administrators, aligning with the chapter's mission of promoting leadership development and professional growth among women.

1.2 Description of the Current System

Currently, the organization uses a manual and fragmented process to manage its leadership training programs. Progress tracking and completion records are kept through spreadsheets and manual data entry, while communication occurs through emails.

Certificates of completion are generated individually using templates, and progress reports require manual compilation. This results in:

- Time-consuming administrative work.
- High potential for data entry errors or missing records.
- Difficulty monitoring trainee engagement and progress.
- Limited visibility into overall program performance.

The lack of automation hinders the organization's ability to track real-time participation and generate accurate, on-demand reports, making the management process inefficient and inconsistent.

1.3 Scope of the System

The system will:

- Provide a secure online platform for both administrators and trainees.
- Allow administrators to create and manage training tracks, modules, and quizzes.
- Allow trainees to view assigned modules, track progress, and receive certificates upon completion.
- Generate and send reports automatically to administrators based on user activity.
- Support automated reminders for inactivity and progress milestones.
- Offer both web and mobile access to ensure usability across devices.

The system will not include unrelated features such as financial accounting, external event scheduling, or social networking functions. Its focus remains solely on leadership development tracking and progress reporting.

1.4 Fundamental Assumptions

The development of the software is based on the following assumptions:

1. All users (administrators and trainees) will have internet access and basic device literacy.
2. Administrators will have permission-based access to manage users and training materials.
3. The organization will maintain reliable hosting and secure cloud storage for the database.
4. The system may integrate with third-party platforms such as LinkedIn Learning or Coursera if needed for leadership curriculum support.
5. User data privacy and security will comply with organizational and data protection standards.

1.5 System Fit and Organizational Context

The Leadership Academy Tracking Software aligns with the NCBW's mission to foster leadership and empower women through structured professional development. The system will:

- Integrate seamlessly into existing administrative workflows.

- Improve record management, trainee accountability, and communication.
- Reduce the need for manual tracking and paper-based processes.
- Enable faster, data-driven decision-making regarding program performance.

By digitizing and automating leadership program management, the software directly supports the organization's long-term goal of operational excellence and scalability.

1.6 Benefits of the System

The system will provide multiple organizational and user-level benefits, including:

Efficiency: Automates routine administrative tasks such as data entry, progress monitoring, and certificate generation, reducing time and effort.

Transparency: Provides both trainees and administrators with real-time visibility into progress, activity status, and completion records.

Accessibility: Accessible from both web and mobile devices, ensuring users can engage with the program anytime, anywhere.

Accuracy: Centralized data storage minimizes human error and ensures consistent recordkeeping across users and modules.

Engagement: Automated reminders and user-friendly interfaces encourage trainees to remain active and complete their training.

Organizational Growth: Supports the vision of developing empowered leaders by providing measurable insights into training outcomes and overall performance.

1.7 Design Goals

- The design goals of the Leadership Academy Tracking Software focus on creating a simple, reliable, and user-friendly system that supports both trainees and administrators. The system is designed to minimize manual work by automating tasks such as progress tracking, report generation, certificate creation, and reminder emails.
- Security is a major goal, with strong measures to protect user data through encryption and controlled access based on user roles.
- The software is also built for scalability, allowing it to support more users, courses, and records as the program grows. In addition, it aims to be accessible on both desktop and mobile devices so users can access the platform at any time.

- Finally, the system emphasizes maintainability, using modular code and clear documentation to make updates, improvements, and troubleshooting straightforward. These goals ensure the system remains efficient, secure, and easy to use for all members of the organization.

1.8 References

- The design of the Leadership Academy Tracking Software is based on requirements and specifications outlined in our [Deliverable 1](#) and [Deliverable 2](#), which provided the foundation for the system's purpose, user roles, and functional requirements. The Requirements Specification document from Deliverable 2 served as the main reference for mapping system features, database structures, and user interactions to design elements.
- The Requirements Specification document from Deliverable 2
 - Mapping System Features, Database Structures, and User Interactions to Design Elements
- Requirements Specifications outlined in Deliverable 1 and 2
- Additional references include customer feedback gathered during early meetings with the [National Coalition of 100 Black Women, Queen City Metropolitan Chapter](#)
- SCORM is a set of technical standards for eLearning software products and is the de facto industry standard for eLearning interoperability. <https://scorm.com/>
- MySQL: MySQL is an open-source, fast, and reliable relational database management system used to store, manage, and organize data for websites and applications. https://www.w3schools.com/MySQL/mysql_intro.asp
- PHP: PHP is a server-side scripting language used to build dynamic web pages and power major platforms like WordPress. https://www.w3schools.com/php/php_intro.asp
- LMS (Learning Management Systems): An LMS is a web-based software system that helps organizations create, deliver, and manage learning content while tracking student participation and performance. [LMS](#)
- WCAG 2.1: WCAG 2.1 is a set of guidelines that explain how to make web content more accessible and usable for people with a wide range of disabilities. <https://www.w3.org/TR/WCAG21/>
- JWT: A JSON Web Token (JWT) is a compact, digitally signed token used to securely transmit information between parties as a JSON object. <https://www.jwt.io/introduction#what-is-json-web-token>
- AES-256: AES-256 is a strong symmetric encryption method that uses a 256-bit key to turn your plain data into an unreadable, secure cipher. [AES-256](#)
- TLS 1.3: TLS 1.3 is the latest version of Transport Layer Security that provides faster, stronger, and more modern protection for data sent over the internet. <https://www.fortinet.com/blog/business-and-technology/tls-is-here-what-this-means-for-you>
- Bcrypt: Bcrypt is a secure password-hashing algorithm that turns a password into a unique, irreversible hash using salt and multiple rounds to protect against dictionary and brute-force attacks. [Bcrypt](#)

2. System Requirements Specification [Modified from Del 2]

2.1 User Characteristics

The Leadership Academy Tracking Software will be used by members of the National Coalition of 100 Black Women, Queen City Metropolitan Chapter, and supporting administrative staff. Users of the system fall into several categories, each with unique characteristics and technical needs.

End-Product Customers (Leaders and Members)

- Roles: President, Vice Presidents, Treasurer, Secretaries, Chaplain, Parliamentarian, and general members enrolled in leadership development.
- Education/Skill Level: Most users are professional with varying backgrounds, often with post-secondary education. While they have strong leadership, communication, and organizational skills, their technical expertise may range from beginner to intermediate.
- Experience: Users may have prior exposure to some tools (like Microsoft Office or Zoom), but not necessarily advanced technical systems.
- Constraints: The software must be intuitive and user-friendly, requiring minimal technical training. Clear navigation, simple data entry forms, and automated tracking features are some examples.

Operators (Administrative Users)

- Roles: Chapter administrators, program coordinators, and committee chairs responsible for managing training records, scheduling, and reporting.
- Education/Skill Level: Moderate computer literacy with experience in data entry, record keeping, and using online platforms.
- Experience: Comfortable with spreadsheets, scheduling tools, and file management.
- Constraints: The system must support batch uploads, detailed reporting, and straightforward record management to lower administrative overhead.

2.2 Functional Requirements

The following section defines the functional behavior of the system, including inputs, processing rules, and expected outputs for each feature.

2.2.1 Generate Certificate of Completion

Description:

The system shall automatically generate a printable certificate of completion after a trainee successfully finishes all modules in a track.

2.2.1.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee ID	Unique identifier of trainee	Must exist in database	Integer	Yes
track ID	Identifier for completed track	Must correspond to finished track	Integer	Yes

2.2.1.2 Outputs

- A PDF certificate file containing trainee name, track title, and completion date.
 - Certificate emailed automatically to the trainee.
-

2.2.2 Send Inactivity Reminder Email

Description:

The system shall automatically email a reminder to trainees who have not completed any module within seven days.

2.2.2.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee ID	Unique trainee record	Must exist	Integer	Yes
last activity date	Date of last module activity	Must be ≥ 7 days old	Date	Yes

2.2.2.2 Outputs

- Email reminder sent to trainee's registered address.
 - Log entry created for audit purposes.
-

2.2.3 Send Progress Report to Administrator

Description:

The system shall email administrators a progress report when triggered by completion or inactivity conditions

2.2.3.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee ID	Identifier of trainee	Must exist	Integer	Yes
trigger event	progress condition (completion, inactivity)	Must match predefined conditions	String	Yes

2.2.3.2 Outputs

- PDF or HTML progress report emailed to designated administrator.
-

2.2.4 Review Completed Modules

Description:

The system shall allow trainees to revisit and view previously completed modules without altering their completion status.

2.2.4.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee ID	User requesting access	Must exist	Integer	Yes
module ID	Identifier for completed module	Must be marked completed	Integer	Yes

2.2.4.2 Outputs

- Module content displayed in read-only mode.
 - Completion status remains unchanged.
-

2.2.5 Generate Video Transcripts

Description:

The system shall automatically generate text transcripts for all videos uploaded by administrators.

2.2.5.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
video file	Uploaded training video	Must be valid file format (MP4/AVI)	File	Yes
admin ID	Uploader identifier	Must have upload permissions	Integer	Yes

2.2.5.2 Outputs

- Transcript (.txt or .vtt) generated automatically and linked to the video.
 - Transcript available for download or review.
-

2.2.6 Create Quizzes

Description:

The system shall allow administrators to create quizzes for any module.

2.2.6.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
admin ID	Creator identifier	Must have quiz permissions	Integer	Yes
module ID	Associated module identifier	Must exist	Integer	Yes
question set	Collection of questions and answers	Cannot be empty	JSON/Text	Yes

2.2.6.2 Outputs

- Quiz saved and linked to its module.
 - Confirmation message displayed to admin.
-

2.2.7 Reset Inactive Progress

Description:

The system shall automatically reset a trainee's progress after 30 days of inactivity, unless the track is already complete.

2.2.7.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee ID	Identifier of trainee	Must exist	Integer	Yes
last activity date	Last logged activity date	Must be ≥ 30 days old	Date	Yes

2.2.7.2 Outputs

- Trainee progress reset to 0%.
 - Email notification sent to trainee.
-

2.2.8 Create Trainee Account

Description:

The system shall allow administrators to create trainee accounts through the website and send login credentials to the new user's email.

2.2.8.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee name	Full name of trainee	Cannot be blank	String	Yes
email	Valid email address	Must match email pattern	String	Yes
role	Account type (trainee/admin)	Must match predefined roles	String	Yes

2.2.8.2 Outputs

- Account created and stored in database.
 - Email sent with temporary password and login URL.
-

2.2.9 Login for Administrators and Trainees

Description:

The system shall allow both administrators and trainees to securely log in using their registered email and password credentials.

2.2.9.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
email	User's registered email	Must match existing user record	String	Yes
password	User password	Must match stored bcrypt hash	String	Yes

2.2.9.2 Outputs

- Successful login redirects to corresponding dashboard (Admin → Admin Dashboard, Trainee → Member Dashboard).
 - Error message displayed for invalid credentials or locked accounts.
 - JWT session token generated and stored temporarily.
-

2.2.10 Reset Password

Description:

The system shall allow users to reset their password by verifying their identity through email.

2.2.10.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
email	Registered email	Must exist in database	String	Yes

2.2.10.2 Outputs

- Password reset link sent via email.
 - New password securely hashed and stored after reset confirmation.
-

2.2.11 Add/Delete Administrator Accounts

Description:

The system shall allow authorized administrators to create and delete administrator accounts.

2.2.11.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
admin ID	Authorized admin's ID	Must have role permissions	Integer	Yes
new admin info	Full name, email, password	Must be unique	JSON/String	Yes

2.2.11.2 Outputs

- New administrator added or removed from database.
 - Confirmation message logged and displayed.
 - Email notification sent to new or removed admin.
-

2.2.12 Delete Trainee Accounts

Description:

The system shall allow administrators to delete trainee accounts that are inactive or no longer members.

2.2.12.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee ID	Trainee account identifier	Must exist	Integer	Yes

2.2.12.2 Outputs

- Trainee record removed from database.
 - Confirmation displayed and logged.
 - Related progress and report data archived or deleted.
-

2.2.13 View Training Modules

Description:

The system shall allow trainees to view available modules in their assigned track.

2.2.13.1 Inputs

Field Name	Field Description	Validation	Type/Length	Req'd
trainee ID	Identifier of logged-in trainee	Must exist	Integer	Yes

2.2.13.2 Outputs

- List of module cards displayed with title, status, and access links.
 - Locked modules visually indicated with lock icons.
-

2.2.14 Logout

Description:

The system shall allow authenticated users to securely log out of the system.

2.2.14.1 Inputs

Input: active session token

2.2.14.2 Outputs

- JWT token invalidated.
 - User redirected to login page.
 - “Successfully logged out” message displayed.
-

2.3 Non-Functional Requirements

2.3.1 Availability and Reliability

The system will remain available at least 99% of the time to ensure users can access it whenever needed. Any scheduled maintenance will take place during low-traffic hours and be communicated to administrators and trainees in advance, along with the expected downtime and restoration period. Backup and recovery measures will be in place to prevent data loss and minimize disruptions if unexpected issues occur. These measures ensure that core system functions remain reliable and available to users.

2.3.2 Platform Compatibility

The system will function reliably across major web browsers (such as Chrome, Safari, Firefox, and Edge) and be accessible through both iOS and Android mobile devices. The interface will adjust smoothly to different screen sizes and device types so users have a consistent experience whether on desktop or mobile. This allows trainees to review modules and administrators to manage accounts or tracks from any device.

2.3.3 Data Integrity and Privacy

All data stored and shared through the system will remain accurate, protected, and confidential. Sensitive information, including login credentials, progress records, and certificates will be encrypted both when stored and when transmitted. Regular data validation, access controls, and system backups will help maintain the integrity and privacy of all NCBW member data in line with organizational standards.

2.3.4 Usability and Accessibility

The interface will be clear, intuitive, and easy to navigate, allowing users to complete essential tasks with minimal effort. Common actions such as logging in, viewing progress, and managing tracks are designed to require minimal navigation steps. The design will also meet WCAG 2.1 accessibility standards to ensure that members of all technical skill levels and abilities can use the platform effectively.

2.3.5 Performance and Responsiveness

Pages and features will load within two seconds or less under normal conditions. The system will remain smooth and stable even when several users are active at once. Performance optimization techniques such as caching and efficient database queries will be used to ensure fast response times.

2.3.6 Scalability and Capacity

The system is designed to support up to approximately 100 concurrent users under normal operating conditions. Its design will allow for easy scaling in the future, enabling other NCBW chapters to adopt the platform if desired. Regular load and stress tests will be conducted to verify that performance remains reliable as usage increases and more tracks or administrators are added.

2.3.7 Maintainability

The system will be built with clean, modular code that is well-documented and easy to update. Future improvements, bug fixes, and version changes will be simple to implement without affecting existing functionality. Consistent version control, clear documentation, and separation of front-end and back-end components will support long-term maintenance by future administrators or developers maintaining the system.

2.3.8 Security and Compliance

Access to the system will be controlled through secure authentication and authorization processes that align with NCBW policies. The software will use AES-256 encryption, TLS 1.3 secure connections, and bcrypt for password hashing. Security reviews and vulnerability testing will be performed regularly to identify and address risks. These controls ensure the safety of all user accounts and protect administrative functions such as creating or modifying modules, managing reports, and adding new tracks.

3. Access Control and Security Decisions

3.1 Access Matrix

Role	View Training Content	Create/Edit Courses	Manage Reports	Create Accounts	View own Progress	Manage Certificates
Trainee	✓			✓	✓	
Administrator	✓	✓	✓	✓	✓	✓
Role	See All Trainee Progress	View & Download Certificates				
Trainee		✓				
Administrator	✓	✓				

3.2 Authentication Mechanism

All users must authenticate through a secure login system using member ID and password credentials. The system enforces the following security measures:

- Passwords are stored using bcrypt hashing with salt
- Passwords must meet complexity requirements:
 - 8-28 characters in length
 - At least one uppercase Letter
 - At least one lowercase Letter
 - At least one number
 - At least one special character (! @ # \$ % * () =)
- Password reuse is restricted per user account through password history checks
- JWT (JSON Web Tokens) are used for secure, stateless session management
- Session tokens are invalidated upon logout or when security anomalies are detected

3.3 Data Encryption

- Sensitive data (including personal information, training progress, and login credentials) is protected using encryption both at rest and in transit.
 - Data in transit is secured using TLS 1.3
 - Passwords are securely hashed using bcrypt
 - Database-stored sensitive information is protected through application-level encryption practices where applicable
- Access to sensitive configuration values and secrets is restricted to authorized system processes.

3.4 Security and Privacy

- The system ensures that all stored and transmitted data remains accurate, secure, and confidential.
- Access control mechanisms are implemented to restrict data access based on user roles. The system follows standard security practices, including authentication, authorization, and regular validation of user inputs.
- Security and privacy measures are reviewed periodically to identify and mitigate potential risks.

4. Design Decisions

4.1 User Interface

The user interface is designed as a role-based dashboard system where content and available actions vary depending on whether the user is a trainee, instructor, or administrator.

The design prioritizes simplicity and usability by:

- Providing a centralized dashboard for navigation
- Minimizing the number of steps required to access core features
- Maintaining consistent layout structures across all user roles

Trainees are presented with course enrollment and progress tracking while instructors and administrators have access to management tools such as course creation, user management, and reporting features.

4.2 Data Presentation

The system presents trainee data through a structured dashboard designed to provide clear visibility into training progress and required coursework.

Key information such as the trainee's name, selected leadership track, and overall completion status is displayed prominently. Completion status is represented both numerically (percentage) and visually using progress indicators.

Training modules are presented using a card-based layout to improve readability and allow users to quickly identify module status and requirements.

The system uses structured visual hierarchy and status indicators to support efficient interpretation of training progress and system state.

4.3 Exception Handling

The system implements exception handling mechanisms to ensure stability and provide user feedback when errors occur.

Validation is performed on both frontend and backend to ensure required data is correctly entered before processing requests.

Authentication failures, invalid inputs, and unauthorized access attempts are handled securely, with appropriate feedback returned to the user without exposing system-level details.

System safeguards are in place to prevent unauthorized access to restricted content and ensure data integrity during unexpected or invalid operations.

4.4 Languages and Rules

The system is built using the following technologies:

- Python is used as the primary backend programming language.
- Django is used as the backend web framework, built on top of Python, to handle authentication, business logic, and database operations.
- SQL is used for structured data storage to manage trainee records, track progress, and support reporting functionality.
- React and JavaScript are used for frontend development to create a responsive and interactive user interface.
- Node.js is used as the runtime environment for frontend development tooling, including package management and running the React development server.

5. Architectural/Detailed Design

5.1 System Overview

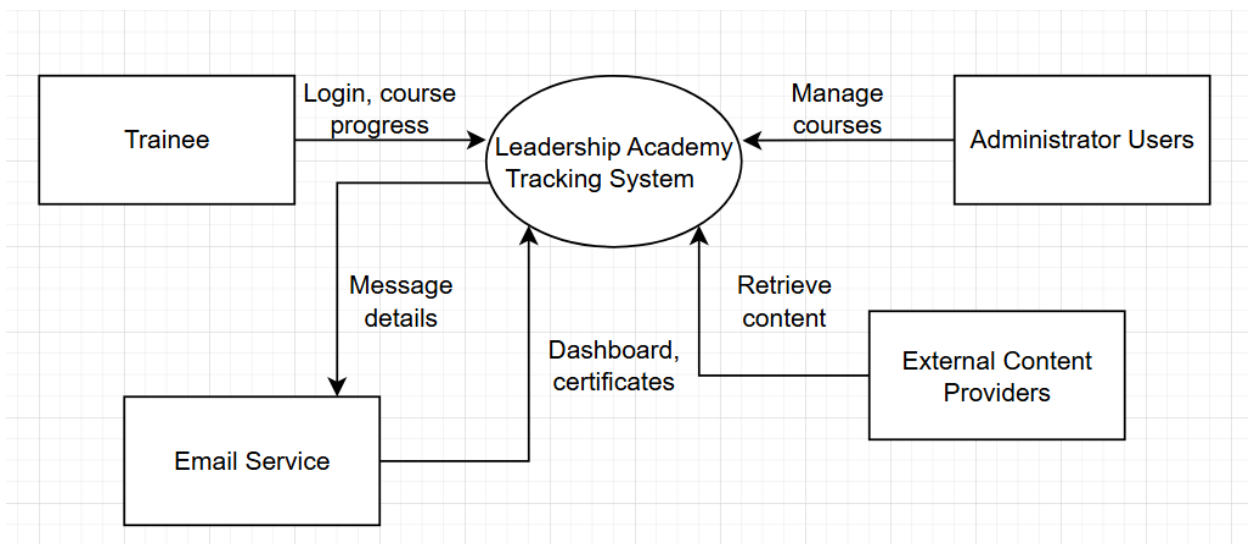
The Leadership Academy Tracking Software is a web-based system designed to centralize and automate the management of leadership training for the National Coalition of 100 Black Women, Queen City Metropolitan Chapter.

The system replaces manual processes such as spreadsheets, email tracking, and manual reporting with an integrated digital platform.

The system is composed of the following core subsystems:

- Authentication and User Management: Handles login, registration, password reset functionality, and role-based access control for trainees and administrators.
- Training Management: Supports creation and management of leadership tracks, modules, quizzes, and associated training content.
- Progress Tracking: Records and updates trainee completion status, tracks activity, and allows both trainees and administrators to review progress.
- Certificate Generation: Produces certificates upon successful completion of training tracks.
- Notification System: Manages automated email reminders and system alerts, including inactivity notifications and progress updates sent to administrators.

All subsystems interact with a centralized database to ensure data consistency and integrity across the platform.



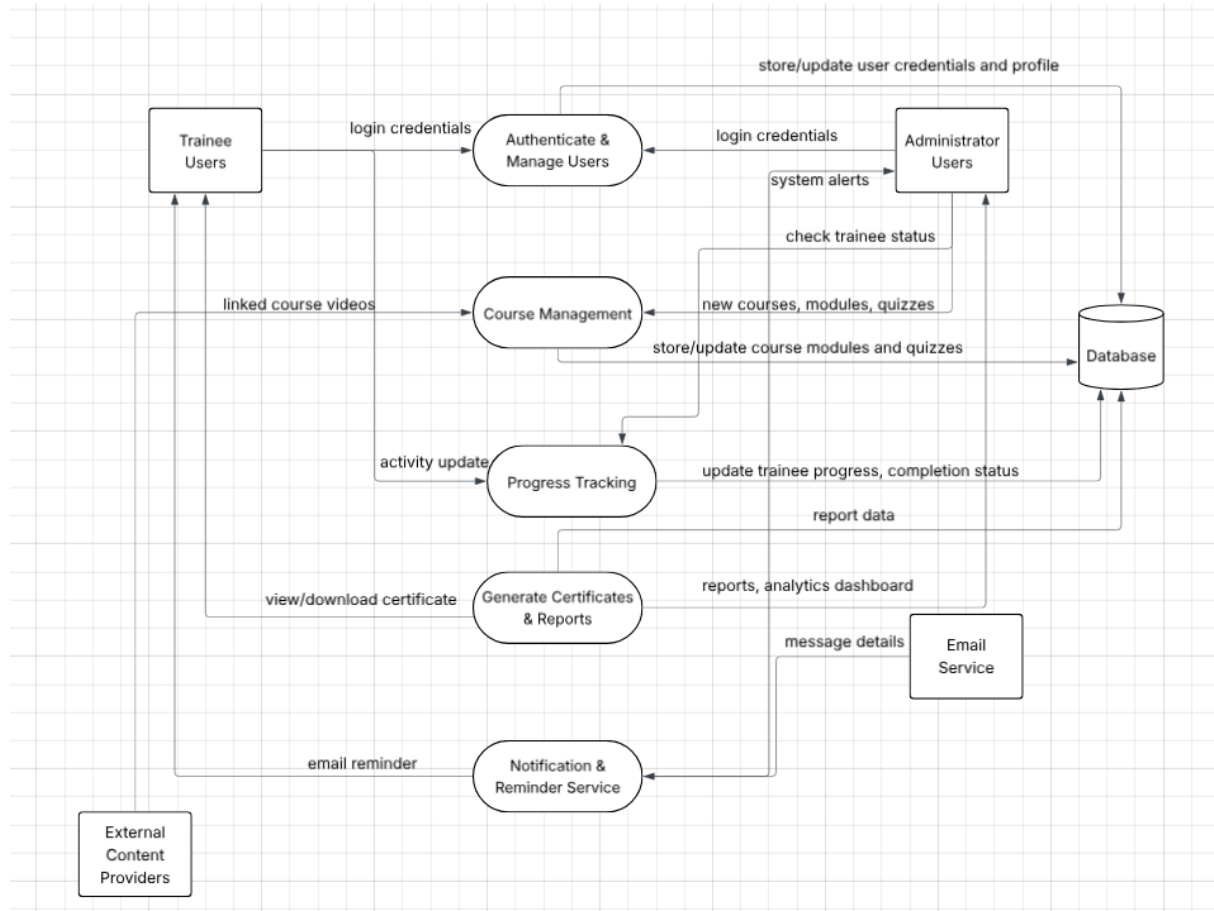
5.2 Subsystem Design

Functional Subsystems:

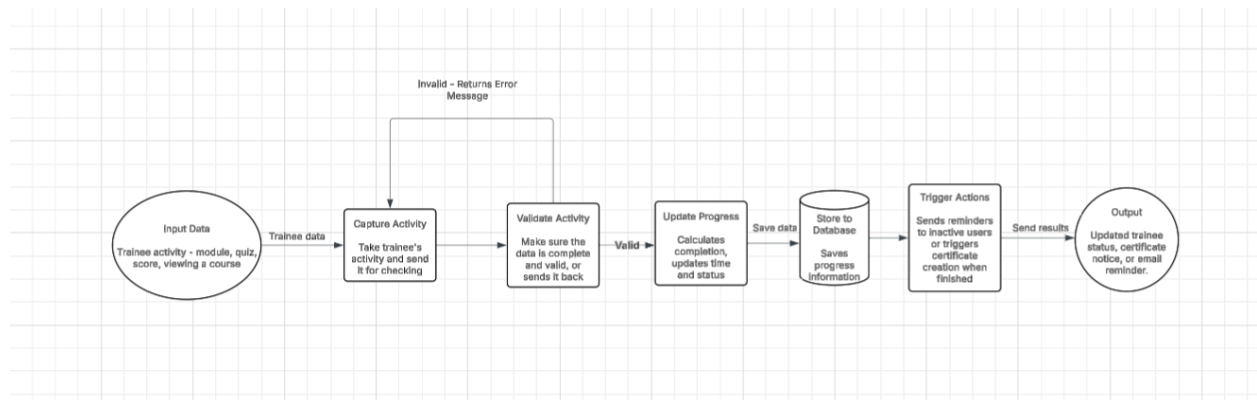
In addition to the layered architecture, the backend is organized into functional components that support key system features:

- **Authentication Module:** Handles login, logout, password reset, and user session management using secure authentication mechanisms.
- **Training Management Module:** Manages courses, modules, quizzes, and associated training content.
- **Progress Tracking Module:** Tracks user activity, completion status, and allows retrieval of progress data for both trainees and administrators.
- **Notification Module:** Sends automated emails such as inactivity reminders and progress updates using scheduled background tasks.
- **Reporting Module:** Generates progress reports for administrators based on trainee data stored in the system.
- **Certificate Module:** Generates and manages certificates upon completion of training programs, including storage and retrieval.
- **Transcript/Content Processing Module:** Supports generation or retrieval of video transcripts associated with training materials.

Level 1 diagram:



Level 2 diagram:



5.3 Persistent data management

Database Selection, Design, and Data Schemes

The Leadership Academy Tracking Software uses a centralized relational database (MySQL) to store and manage all system data securely and efficiently.

The database supports all major system functions including authentication, course management, progress tracking, notifications, reporting, and certificate generation.

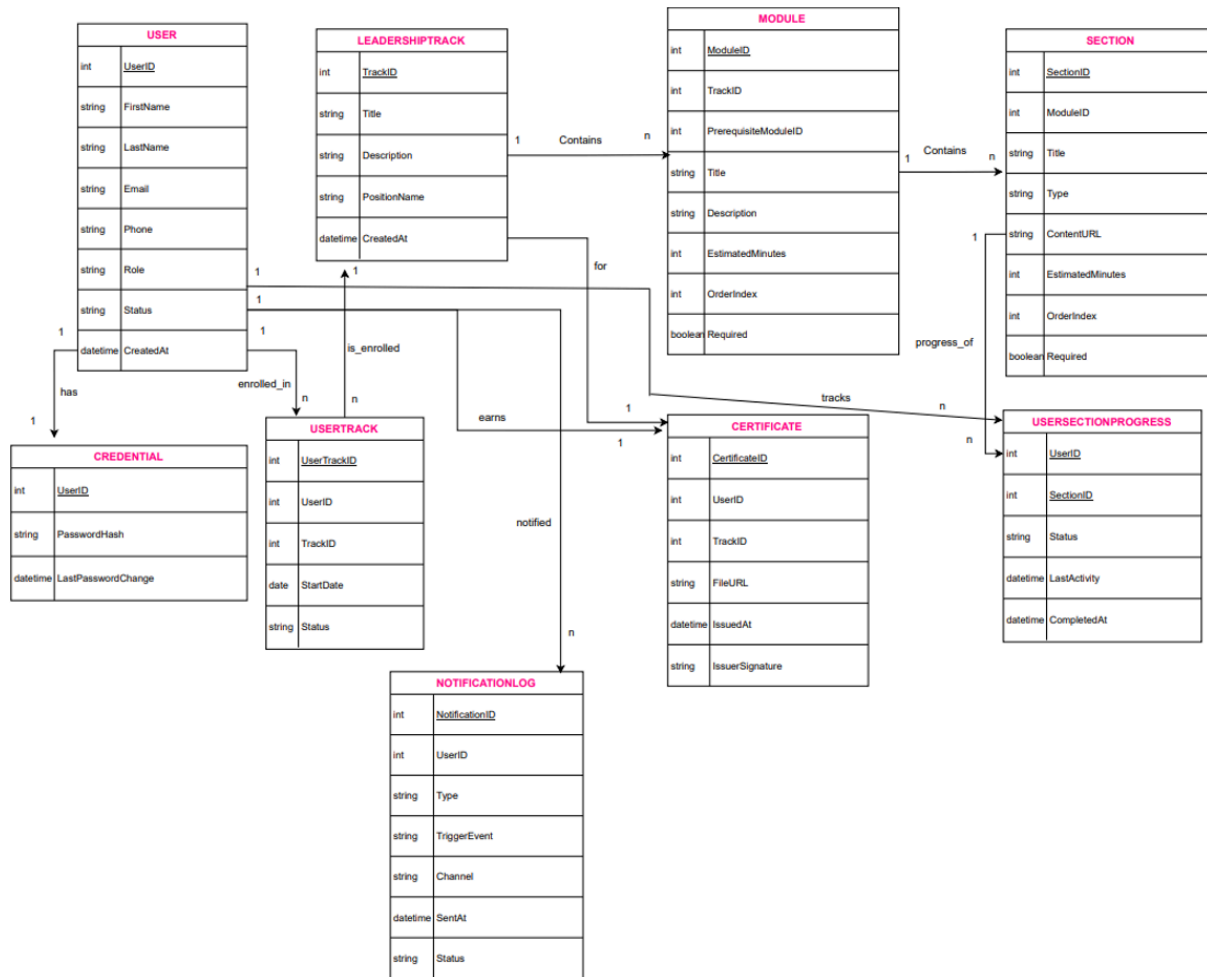
Sensitive data such as passwords and personal information is protected using secure hashing (bcrypt) and access control mechanisms.

Data Stored in the Database Includes:

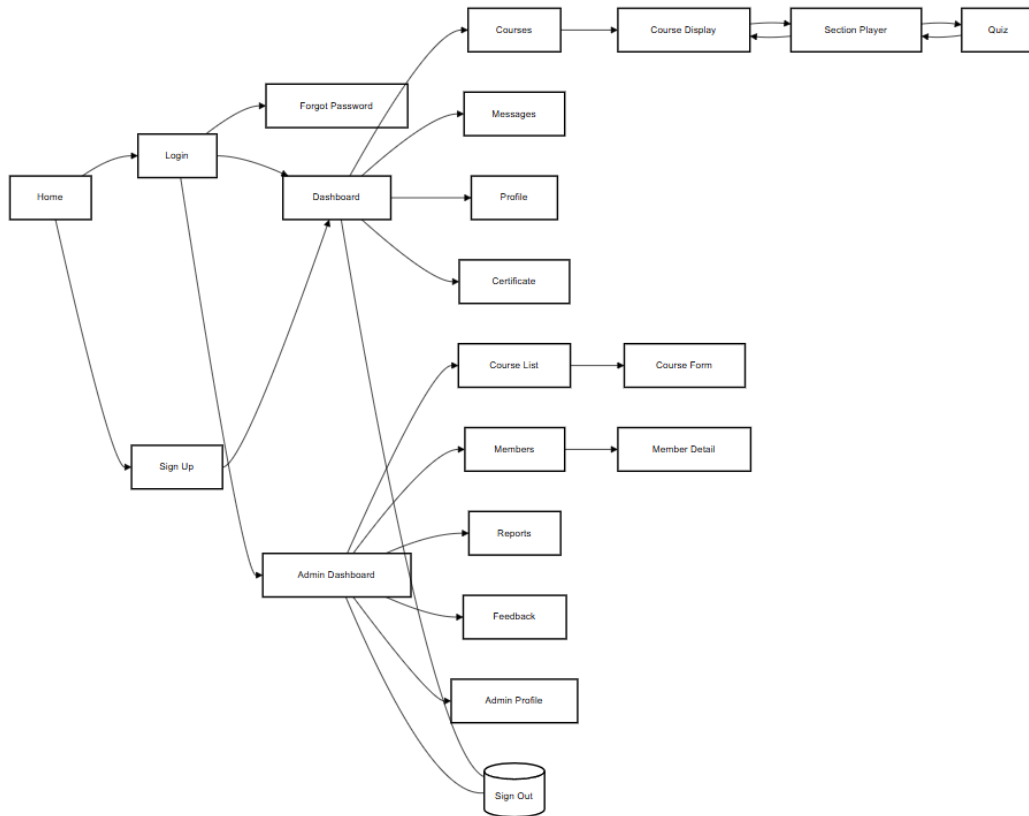
- User Information: user ID, name, email, role, and authentication credentials.
- Training Data: leadership tracks, modules, and course content metadata.
- Progress Data: completion status, timestamps, activity history, and last activity dates used for inactivity tracking.
- Certificates: issued certificate records, metadata, and associated file references.
- Notifications: system-generated messages, inactivity reminders, and email logs.
- Reports: aggregated trainee progress data used for administrative reporting.
- Transcripts: stored or referenced transcript data associated with training videos.

Each user is assigned a unique User ID, which is used as a primary key and referenced across related tables to maintain relational integrity.

The ER diagram provides a complete overview of the system's relational database, including all primary entities and the relationships used to manage users, training content, progress tracking, and system-generated actions.



This Site Navigation Diagram shows how users move through the website. Visitors can log in or sign up, and are then directed to a dashboard depending on their role. They can enter different areas of the site depending on which dashboard they are in as well. The diagram illustrates the main pages and the simple navigation flow that will be present.



5.4 Project Structure

The system follows a structured separation between frontend and backend components.

- The frontend is organized using a React-based structure, with components, pages, and services grouped into modular directories for maintainability.
- The backend is structured as a Django project with multiple apps responsible for handling authentication, training management, and progress tracking.
- Shared configuration files and environment settings are separated to support scalability and ease of deployment.

This modular structure improves code organization, maintainability, and collaboration among developers.

6. Requirement Traceability Matrix

Project Information						
Project Name:	Leadership Academy Tracking Software			Initial Creation Date:	10/30/25	
Req Id	Requirement Name	Ref Id	Design Section	Implementation	Validation Method	Status
1	Generate Certificate of Completion	2.2.1	5.1, 5.3	Django backend (certificate module)	Test	Complete
2	Send Inactivity Reminder Email	2.2.2	5.1, 5.2	Scheduled Django management command executed via OS script	Test	Complete
3	Send Progress Report to Administrator	2.2.3	5.1, 5.2	Django reporting + email service	Test	Complete
4	Review Completed Modules	2.2.4	5.2	React frontend + Django API	Inspection	Complete
5	Generate Video Transcripts	2.2.5	5.2	Backend processing/transcript service	Test	Complete
6	Create Quizzes	2.2.6	5.2	Django admin + database models	Test	Complete
7	Reset Inactive Progress	2.2.7	5.1, 5.2	Scheduled Django management command executed via OS script	Test	Complete
8	Create Trainee Account	2.2.8	5.2	Django authentication system	Test	Complete
9	Login	2.2.9	5.1, 5.2	Django auth + JWT	Test	Complete
10	Reset Password	2.2.10	5.2	Django password reset system	Test	Complete
11	Add/Delete Administrator Accounts	2.2.11	5.2	Django admin management	Test	Complete
12	Delete Trainee Accounts	2.2.12	5.2	Django user management	Test	Complete
13	View Training Modules	2.2.13	5.2	React frontend + API	Inspection	Complete
14	Logout	2.2.14	5.2	JWT session handling	Test	Complete

7. System Deployment

7.1 System Requirements

The system requires the following environment:

Core Requirements:

- Operating System: Windows, Linux, or macOS
- Programming Language: Python 3.12.x
- Backend Framework: Django
- Frontend Runtime: [Node.js](#) (required for running the frontend development server)
- Database: MySQL
- Web Browser: Modern browser (Chrome, Firefox, Edge)
- Package Managers:
 - pip (Python dependencies)
 - npm ([Node.js](#) dependencies)

Development and Deployment Tools:

- Git (for version control and repository management)
- Virtual environment (venv) for dependency isolation

Access Requirements:

- GitHub account (required for accessing & cloning the repository, if applicable)

7.2 Repository Setup

To obtain the system source code:

- Open a command line in the directory you wish to download the project to
- Clone the repository
 - `git clone -b aifa-login https://github.com/WinthropUniversity/project-2025-2026-cabepp/`
 - Note: You will need to have a Github account in order to download the repository.

After cloning, navigate to the project directory:

- `cd <project_directory>`

7.3 Backend Setup

The backend of the system is implemented using Django and handles business logic, authentication, and database interactions.

Setup steps:

1. Navigate to the backend directory in the project folder
2. Open a command line/terminal
3. Create and activate a virtual environment:
 - a. `python -m venv venv`
 - i. `venv\Scripts\activate` (Windows)
 - ii. `source venv/bin/activate` (macOS/Linux)
4. Install required dependencies:
 - a. `pip install -r requirements.txt`
5. Apply database migrations:

- a. `python manage.py makemigrations`
 - b. `python manage.py migrate`
 - c. (Note: Running migrations will automatically create a database file for you)
6. Create an administrative user:
 - a. `python manage.py createsuperuser`
7. Start the backend server:
 - a. `python manage.py runserver`

7.4 Frontend Setup

The frontend of the system is implemented using JavaScript and is served through a development server.

Setup steps:

1. Navigate to the frontend directory
2. Open a command line/terminal
3. Install dependencies:
 - a. `npm install`
4. Start the frontend development server:
 - a. `npm run dev`

The frontend communicates with the Django backend through API endpoints.

7.5 Running the Full System

To run the full system, both backend and frontend servers must be active simultaneously.

1. Start the Django backend server from the backend project directory
 - a. (venv)
 - i. `venv\Scripts\activate` (Windows)
 - ii. `source venv/bin/activate` (macOS/Linux)
 - b. `python manage.py runserver`
2. In a separate terminal, start the frontend server:
 - a. `npm run dev`
3. Access the application through the following development URLs
 - a. <http://localhost:5173> (frontend)
 - b. <http://127.0.0.1:8000/admin> (backend)

8. System Configuration & Administration

8.1 System Access and Roles

The system implements role-based access control to ensure that users can perform only the actions appropriate to their role. The primary user roles are Trainee, Instructor, and Administrator.

Role	Key Permissions / Responsibilities
Trainee	- Access assigned leadership track and modules - Track progress and module completion - Generate certificate upon completion of all required modules
Instructor	- Create and manage course modules - Monitor trainee progress and completion - Monitor trainee progress and completion
Admin	- Manage users and assign roles - Configure system settings - Oversee overall system operations

Notes:

- Permissions are enforced by the backend and reflected in the UI, so users only see actions relevant to their role.
- Role-based access ensures security, prevents unauthorized changes, and supports smooth operational workflows.

8.2 Course and Certificate Management

Administrators and instructors have access to tools for managing courses, modules, and certificates. Role-based access ensures that only authorized users can perform these actions.

Course and Module Management:

- Create, edit, or remove courses and modules.
- Assign modules to specific leadership tracks.
- Configure module order and prerequisites.
- Monitor trainee progress through assigned modules.

Certificate Management:

- Certificates are automatically generated once a trainee completes all required modules in their selected track.
- Administrators can configure certificate templates, including formatting and content.
- Certificates can be reviewed or reissued if necessary.

8.3 System Configuration and Scheduled Tasks

Administrators are responsible for maintaining system configuration and ensuring that all components operate correctly across environments

Backend Configuration:

- The backend is managed through the Django framework.
- Configuration settings (e.g., database, allowed hosts, debug mode) are defined in the settings.py file located in the “core” directory.

Frontend Configuration:

- The frontend application communicates with the backend API.
- Administrators must ensure that API endpoints are correctly configured and aligned with the backend server

Environment Considerations:

- Development and production environments may require different configurations (e.g., database type, debug settings, URLs).
- For initial setup instructions, refer to Section 7.

Scheduled Tasks and Automation:

- The system includes automated maintenance tasks to support administrative operations. One such task is the inactivity monitoring script.

Inactivity Task Script Setup (Windows - Task Scheduler):

1. Ensure the backend environment is properly configured and accessible.
2. Search for and open “Task Scheduler.”
3. Click on the Action Tab and select “Create a Basic Task.”
 - a. Name: Inactivity Task (or whatever you wish to name it) (+ An optional description)
 - b. Trigger: Daily
 - i. Start: (change as needed)
 - ii. Time: 12:00 AM (recommended)
 - iii. Recur every: 1 Days
 - c. Action: Start a program
 - i. Set the path to the “*run_inactivity_tasks_windows.bat*” batch file, located in the backend directory of the project folder (e.g., *C:\path\to\inactivity_tasks_windows.bat*).
 - ii. (leave the optional fields blank)
4. Edit the Properties of the newly created task
 - a. Right click the task and select “Properties.”
 - b. In the General Tab
 - i. Select the “Run whether the user is logged on or not” radio button.
 - ii. Select the “Run with highest privileges” checkbox.
 - iii. (Note: you will be asked to enter the account information (username and password) for your machine before confirming these changes).
5. Save the task and verify it runs successfully.
 - a. (Note: Test “*run_inactivity_tasks_windows.bat*” manually by double-clicking on the file)

Inactivity Task Script Setup (macOS/Linux - Cron Jobs):

1. Open a terminal
2. Edit the crontab file by running:
 - a. `crontab -e`
3. Add a newline to schedule the task:
 - a. `0 0 * * * /path/to/project/inactivity_tasks_unix.sh`
 - i. (Runs the script daily at midnight)
4. Save and exit the editor.
5. Ensure the script has execute permissions:
 - a. `chmod +x /path/to/project/inactivity_tasks_unix.sh`
6. Verify the script runs successfully.

Notes:

- Detailed startup procedures are described in Section 9.
- Configuration changes should be tested in a development environment before being applied to production.

9. System Operations

9.1 System Startup and Shutdown

To operate the system, both the backend and frontend servers must be started.
For initial setup and installation instructions, refer to **Section 7: System Deployment**.

Startup Procedure:

1. Activate the backend virtual environment:
 - a. `venv\Scripts\activate` (Windows)
 - b. `source venv/bin/activate` (macOS/Linux)
2. Start the Django backend server:
 - a. `python manage.py runserver`
3. In a separate terminal, start the frontend server:
 - a. `npm run dev`
4. Access the application through the frontend URL
 - a. <http://localhost:5173> (frontend)

Shutdown Procedure:

1. To stop the backend server, press CTRL + C in the backend terminal.
2. To stop the frontend server, press CTRL + C in the frontend terminal.

9.2 User Workflows

The system supports three primary user roles: trainees, instructors, and administrators. Each role follows a defined workflow that reflects their interaction with the system.

Trainee Workflow:

- Trainees authenticate into the system and access their assigned leadership track.
- Trainees progress through required modules in a sequential manner, with completion status tracked by the system.
- Upon completing all required modules, the system enables certificate generation.

Instructor Workflow:

- Instructors access the system to create and manage course content.
- They are responsible for maintaining modules and monitoring trainee progress and completion.

Administrator Workflow:

- Administrators oversee system operations, including user management, role assignment, and overall system configuration.
- Admins ensure that trainees and instructors are properly assigned and that the system operates as intended.

9.3 Data Backup

- System data is stored in the database and should be backed up regularly to prevent data loss.
- Manual backup commands:
 - `copy db.sqlite3 D:\backups\leadership_app_db_backup.sqlite3` (Windows)
 - `cp db.sqlite3 ~/backups/leadership_app_db_backup.sqlite3` (macOS/Linux)
- For development environments, backups can be performed by manually copying the database file to a secure location.
- For production environments, it is recommended to implement scheduled database backups and ensure secure storage of all backup files. Automated backup scripts can also be integrated with system schedulers such as Task Scheduler (Windows) or cron jobs (macOS/Linux).

9.4 Monitoring and Logging

The system provides basic logging through the Django framework. During development, errors and system activity are displayed in the terminal where the backend server is running.

Logging can be configured within the Django settings file to capture additional information such as errors, warnings, and user activity. Logs should be reviewed regularly to ensure the system is operating correctly and to identify potential issues.

Monitoring system performance may include:

- Observing server response times
- Checking for errors in logs
- Ensuring both frontend and backend services are running properly

Automated tasks, such as the scheduled `inactivity_tasks` scripts (described in Section 8.3), run in the background to maintain system integrity by processing inactive users. Monitoring these tasks, and verifying their successful execution, is part of the overall system logging and operational oversight.

10. Error Handling & Troubleshooting

10.1 Error Handling Approach

The system uses built-in error handling mechanisms provided by the Django framework to manage runtime exceptions and invalid user actions.

- Input validation is enforced on both frontend and backend.
- Unauthorized access attempts are restricted through role-based access control.
- Errors are handled gracefully to prevent system crashes and maintain data integrity.

10.2 Logging and Error Reporting

The system provides logging capabilities through the Django backend.

- During development, logs and errors are displayed in the terminal running the backend server.
- Additional logging can be configured in the Django settings file to capture warnings, errors, and system activity.

Automated scripts (e.g., inactivity task scripts) generate log files stored in the project's logs directory.

These logs include:

- Execution timestamps
- Task status messages
- Error output (if any)

Administrators should periodically review logs to ensure proper system operation.

10.3 Common Issues and Resolutions

Issue: Backend server fails to start

- Ensure the virtual environment is activated
- Verify all dependencies are installed
- Check for port conflicts (default: 8000)

Issue: Frontend fails to load or connect to backend

- Ensure the frontend server is running
- Verify API endpoint URLs are correctly configured
- Check for CORS or network issues

Issue: Database errors or missing data

- Confirm database configuration in settings.py
- Ensure migrations have been applied
- Verify database service is running (for non-SQLite systems)

Issue: Scheduled tasks not executing

- Verify Task Scheduler or cron job configuration
- Ensure correct user account and permissions are set
- Check script paths and execution permissions
- Review log files generated by the script

Issue: No log files generated

- Confirm logs directory exists or is created by the script
- Verify file write permissions
- Ensure script is executing successfully

10.4 Recovery Procedures

In the event of system failure or data issues, the following recovery steps are recommended:

- Restart backend and frontend services
- Review recent log files for error messages
- Restore the database from a backup if necessary
- Re-run scheduled tasks manually to verify functionality

11. Maintenance & Future Enhancements

11.1 System Maintenance

Ongoing maintenance is required to ensure the system remains secure, reliable, and up to date.

Maintenance activities include:

- Applying updates to the Django framework and dependencies
- Monitoring system logs for errors or unusual activity
- Verifying scheduled tasks are executing correctly
- Ensuring database backups are performed regularly

Administrators should periodically review system performance and address any issues that may impact usability or reliability.

11.2 Routine Maintenance Tasks

The following routine tasks should be performed on a regular basis:

- Review log files for errors and failed processes
- Verify that automated scripts (e.g., inactivity reminders and progress resets) are running as scheduled
- Check database integrity and perform backups
- Remove or archive outdated data if necessary
- Restart services if performance degradation is observed

These tasks help ensure consistent system performance and prevent long-term issues.

11.3 Future Enhancements

The system may be extended with additional features and improvements over time. Potential enhancements include:

- Enhanced reporting and analytics for tracking trainee progress
- Improved user interface and user experience
- Integration with external systems (e.g., email services, authentication providers)
- More advanced notification systems (e.g., SMS or push notifications)

- Role-based dashboards for administrators and instructors
- Deployment to a cloud-based infrastructure for improved scalability

Future updates should be tested in a development environment before being applied to production to ensure system stability.

12. Testing and Validation Summary

12.1 Testing Approach

The system was tested throughout development to ensure that all features function correctly and meet the specified requirements.

Testing was performed incrementally as features were implemented, allowing issues to be identified and resolved early in the development process.

Both manual and functional testing methods were used to validate system behavior.

12.2 Types of Testing Performed

The following types of testing were conducted:

Functional Testing:

- Verified that core features such as user authentication, course progression, certificate generation, and administrative controls operate as intended.

Integration Testing:

- Ensured proper communication between the frontend and backend components, including API requests and data handling.

Database Testing:

- Confirmed that user data, training progress, and system data are stored, retrieved, and updated correctly.

Script Testing:

- Validated automated maintenance scripts, including inactivity reminder emails and progress reset functionality.

User Interface Testing:

- Checked that the frontend displays correctly and provides a usable experience across modern web browsers.

12.3 Results and Validation

Testing results indicate that the system meets its functional requirements and performs as expected under normal operating conditions.

All major features were successfully validated, including:

- User role workflows (trainee, instructor, administrator)
- Training module progression and tracking

- Automated email notifications
- Scheduled maintenance tasks

Any issues encountered during testing were resolved prior to final deployment.

The system is considered stable and ready for use in a production environment, with ongoing monitoring recommended to ensure continued reliability.

13. Glossary

AES-256: One of the encryption algorithms that will be in-place to secure sensitive data (personal information, training data, and login credentials). Uses a 256-bit key for data encryption/decryption.

Amazon Web Services (AWS): A comprehensive cloud computing platform that provides a vast array of services, including computing power, storage, and databases, to businesses of all sizes.

AWS KMS: A key-management service from AWS that enables users to create and control cryptographic keys for encrypting and signing data.

Backend: The part of a system or application that is not directly seen by the user and handles the "behind-the-scenes" functions of the system/application.

Bcrypt: A password-hashing function designed to protect against brute-force attacks by being computationally intensive.

Bit: The smallest unit of information, represented by a 0 or 1.

Ciphertext: Unreadable data that has been encrypted to protect sensitive information.

Computationally Intensive: A process or task that requires a significant amount of processing power, memory, and time to complete due to complex calculations.

Course: A structured collection of learning materials, activities, and assessments within a leadership track. Each course is divided into modules that trainees must complete to progress and earn certification.

Cryptography: The practice and study of techniques for secure communication in the presence of adversarial behavior.

Database: A structured collection of data where login credentials, trainee progress, quiz results, and system activity logs are securely stored.

Decryption: The process of converting ciphertext back into its original readable format (plaintext) using the correct key.

Django: A high-level Python web framework for rapid, backend development.

Encryption: The process of converting data into an unreadable format (ciphertext) using an algorithm and key.

HyperText Markup Language (HTML): the standard language for creating web pages and structuring their content.

Inactivity Reset: The process where a trainee's progress in a track is cleared if no activity occurs for 30 consecutive days, unless the track is already completed.

Interoperability: The ability of computer systems or software to exchange or make use of information.

Issuer Signature: A digital or system-stored signature included on generated certificates to validate authenticity.

JavaScript: A programming language for enabling interactive web pages and dynamically updating content.

Key Management Service: A system for generating, storing, and managing cryptographic keys used for encryption and decryption.

Login interface: The screen that users will see when they access the training program. Consists of a field to enter a username and a field to enter a password along with a button to click if the user has forgotten their password.

Member ID: A unique identifier assigned to each trainee, used for login and tracking progress in the system.

Module: A self-contained unit of learning within a course that focuses on a specific topic or skill. Each module may include videos, readings, quizzes, and other interactive materials that trainees must complete before advancing to the next module or earning certification.

Password hashing: The process of converting a password into a fixed-length, unreadable string of characters using a one-way mathematical function.

Plaintext: Unformatted text or data that is in its original, human-readable form before being encrypted, or after being decrypted.

Python: A high-level general-purpose programming language.

React: An open-source JavaScript library used for building user interfaces.

Section: A smaller unit within a module that organizes related topics, lessons, or activities into manageable parts. Each section contributes to completing the module and helps trainees focus on specific learning objectives before progressing.

Signing Data: A cryptographic process that uses a sender's private key to create a signature for a document or file, ensuring its authenticity, integrity, and non-repudiation.

Structured Query Language (SQL): A standard language for storing, manipulating and retrieving data in databases.

System Boundary: An imaginary line that separates a defined system from its surroundings, helping to identify what is included in the tracking software and what is not.

TLS 1.3: Transport Layer Security protocol. Used to secure communication over a network by providing encryption, authentication, and data integrity. Most commonly seen as HTTPS on websites.

Track: A complete leadership development pathway composed of multiple courses or modules organized around a specific role or competency area. Completing all required modules within a track allows a trainee to earn a certificate of completion and demonstrate mastery of that leadership focus.

Triggering Condition: A system-defined event (completion of a track or trainee inactivity) that automatically causes a system action such as sending an email or generating a report.

User Authentication: The process of verifying a user's login credentials (member ID and password) before granting access to the system.

Video Transcriptions: Captions/Subtitles transcribing the dialogue and audio of a video.

Web Video Text Tracks (WebVTT): Text tracks providing specific text "cues" that are time-aligned with other media, such as video or audio track.

14. Appendix

Appendix A: Project Folder Structure

The following outlines the full directory structure of the Leadership Academy Tracking Software project.

```
project-2025-2026-cabepp/
├── backend/                                # Django backend application
│   ├── accounts/                          # User authentication and account management
│   │   ├── management/
│   │   │   └── commands/                  # Custom Django management commands (scheduled tasks)
│   │   └── migrations/                   # Database migration files for accounts
│   ├── core/                             # Core project configuration and settings
│   ├── notifications/                    # Email and notification handling (inactivity, alerts)
│   │   └── migrations/                   # Database migrations for notifications
│   ├── training/                         # Training modules, progress tracking, and certificates
│   │   └── migrations/                   # Database migrations for training data
│
│   ├── frontend/                         # React frontend application
│   │   ├── components/                  # Reusable UI components
│   │   ├── public/                     # Static public assets (HTML, icons, etc.)
│   │   └── src/                         # Main source code
│   │       ├── api/                    # API communication with backend
│   │       ├── assets/                 # Images, styles, and static resources
│   │       ├── components/             # React UI components
│   │       ├── pages/                  # Page-level components (views/routes)
│   │       └── utils/                  # Utility/helper functions
```

Appendix B: Database Schema (Based on Django Implementation)

User Management & Authentication

User (Django AUTH User Model)

- Purpose: Core authentication system provided by Django.

Field	Type	Description
id	Integer (PK)	Unique user ID
username	String	Login identifier (member ID usage supported)
email	String	Email Address
password	String	Hashed password (Django auth)
is_active	Boolean	Account active status
is_staff	Boolean	Admin permissions
is_superuser	Boolean	Full system access

Profile

- Purpose: Extends User model with roles, status, and organizational metadata.

Field	Type	Description
id	Integer (PK)	Profile ID
user_id	Integer (FK)	Linked user
member_id	String	Unique member login ID
role	String	trainee / instructor / admin
position	String	Leadership position
status	String	pending / active / in_progress / inactive / deleted
phone	String	Contact number
department	String	Department
invited_by	Integer (FK)	User who created account
invited_at	DateTime	Invitation timestamp
activated_at	DateTime	Activation timestamp
deleted_at	DateTime	Soft delete timestamp
inactivity_reminder_sent_at	DateTime	Last reminder sent

created_at	DateTime	Created timestamp
updated_at	DateTime	Updated timestamp

Login Security

- Purpose: Tracks failed login attempts and account lockouts.

Field	Type	Description
id	Integer (PK)	Item ID
module_id	Integer (FK)	Parent module
title	String	Item title
item_type	String	pdf / video / quiz / text / link
file	File	Uploaded file (optional)
external_url	URL	External content link
text_content	Text	Embedded text content
quiz_data	JSON	Quiz structure
due_date	Date	Optional deadline
order	Integer	Display order
is_visible	Boolean	Visibility control
audience_type	String	all / role / member
created_at	DateTime	Created timestamp

Course & Content Structure

Course

- Purpose: Represents a leadership training course created by instructors.

Field	Type	Description
id	Integer (PK)	Course ID
name	String	Course title
code	String	Optional course code
description	Text	Course description
image	URL	Course image

instructor_id	Integer (FK)	Instructor (User with instructor role)
open_date	Date	Start date
due_date	Date	End date
enrollment	Integer	Enrollment count
status	String	Draft / Published
created_by	Integer (FK)	Creator (admin/instructor)
created_at	DateTime	Creation timestamp
updated_at	DateTime	Update timestamp

Module

- Purpose: Groups learning content inside a course.

Field	Type	Description
id	Integer (PK)	Module ID
course_id	Integer (FK)	Parent course
title	String	Module title
order	Integer	Display order
is_visible	Boolean	Visibility toggle
created_at	DateTime	Creation timestamp

Item (Core Learning Content Unit)

- Purpose: Individual learning materials (video, PDF, quiz, text, link).

Field	Type	Description
id	Integer (PK)	Item ID
module_id	Integer (FK)	Parent module
title	String	Item title
item_type	String	pdf / video / quiz / text / link
file	File	Uploaded file (optional)
external_url	URL	External content link
text_content	Text	Embedded text content

quiz_data	JSON	Quiz structure
due_date	Date	Optional deadline
order	Integer	Display order
is_visible	Boolean	Visibility control
audience_type	String	all / role / member
created_at	DateTime	Created timestamp

Access Control Tables

CoursePositionAccess

- Purpose: Restricts course access by leadership position.

Field	Type	Description
id	Integer (PK)	Record ID
course_id	Integer (FK)	Course
position	String	Allowed position

CourseMemberAccess

- Purpose: Grants course access to specific users.

Field	Type	Description
id	Integer (PK)	Record ID
course_id	Integer (FK)	Course
user_id	Integer (FK)	Specific user

ItemPositionAccess

- Purpose: Restricts item-level access by position.

Field	Type	Description
id	Integer (PK)	Record ID
item_id	Integer (FK)	Learning item
position	String	Allowed position

ItemMemberAccess

- Purpose: Grants item access to specific users.

Field	Type	Description
id	Integer (PK)	Record ID
item_id	Integer (FK)	Learning item
user_id	Integer (FK)	Specific user

Progress Tracking

ItemProgress

- Purpose: Tracks user progress for each learning item.

Field	Type	Description
id	Integer (PK)	Progress record
user_id	Integer (FK)	User
item_id	Integer (FK)	Item
status	String	not_started / in_progress / completed
last_activity	DateTime	Last interaction
completed_at	DateTime	Completion timestamp

Notifications

Notification

- Purpose: System and instructor-generated messages.

Field	Type	Description
id	Integer (PK)	Notification ID
user_id	Integer (FK)	Recipient
course_id	Integer (FK)	Related course (optional)
title	String	Notification title
message	Text	Message body
notification_type	String	info / reminder / warning / etc.
is_read	Boolean	Read status
created_at	DateTime	Timestamp

